

PERSONNEL MONITORING SYSTEM

Data Flow Diagram (DFD) Report
Batangas State University – Lipa Campus | CET Department

INTRODUCTION

This document describes the Data Flow Diagrams (DFD) for the Personnel Monitoring System developed for Batangas State University – Lipa Campus, CET Department. The system combines a full-stack web application with a Python-based facial recognition service that uses IP cameras for real-time attendance tracking.

The Level 0 DFD shows the system as a single process with all external entities that interact with it.

External Entity	Data Going IN	Data Going OUT
Admin	Login credentials, personnel info, photo	Dashboard stats, attendance logs, personnel list
Staff / User	Login credentials	Attendance logs (own department only)
IP Cameras	Live RTSP video stream	(none)
Facial Recognition Service	Personnel photos (pull from API)	Detected identity + log entry (push to API)
PostgreSQL Database	Stored records	Retrieved records

The system is broken down into five main processes:

P1 — Authentication

- Input: Username + password from Admin or Staff/User
- Process: Verify credentials against accounts table, generate session token
- Output: Secure HTTP-only session cookie returned to browser
- Data Stores: accounts table (D2), sessions table (D3)

P2 — Personnel Management

- Input: Personnel information + photo upload from Admin
- Process: Create / Update / Delete personnel records and linked user accounts
- Output: Confirmed record, updated personnel list
- Data Stores: personnel table (D1), accounts table (D2)

P3 — Facial Recognition & Attendance Logging

- Input: RTSP video stream from IP cameras (Camera 1: 192.168.1.64, Camera 2: 192.168.1.65)
- Process (Python service): Detect face !' track !' match embeddings !' confirm identity !' POST log to API
- Input to API: Detected employee ID + timestamp (secured via API key)
- Process (API): Toggle TIME IN / TIME OUT, enforce 8-second cooldown to prevent duplicates
- Output: Attendance log entry saved
- Data Store: attendance_logs table (D4)

P4 — Live Monitoring & Dashboard

- Input: HTTP request from Admin/User (auto-refresh every 5 seconds)
- Process: Fetch and aggregate attendance logs and personnel counts per department
- Output: Real-time attendance feed, present/absent statistics per department
- Data Stores: attendance_logs table (D4), personnel table (D1)

P5 — Photo Sync (Recognition Service Bootstrap)

- Input: API key from the local Python service
- Process: Export all registered personnel photos in Base64 format
- Output: Personnel photos downloaded to local recognition service machine
- Data Store: personnel table (D1) — photoUrl field

PERSONNEL MONITORING SYSTEM

Data Flow Diagram (DFD) Report — Continued
Batangas State University – Lipa Campus | CET Department

LEVEL 2 DFD — PROCESS 3: FACIAL RECOGNITION & ATTENDANCE LOGGING

This is the most critical and unique process in the system. It is expanded here to show the sub-processes within the Python facial recognition service.

Step	Sub-Process	Description
P3.1	Frame Capture	Reads continuous RTSP video frames from IP cameras using OpenCV. Auto-reconnects on stream failure.
P3.2	Preprocessing	Applies CLAHE (Contrast Limited Adaptive Histogram Equalization) and adaptive gamma correction to face crops to improve accuracy under poor lighting.
P3.3	Face Detection	Uses RetinaFace (InsightFace buffalo_L model) to detect face bounding boxes and landmarks in each frame.
P3.4	Face Tracking	Uses DeepSORT to assign a consistent Track ID to each detected face across multiple frames, preventing duplicate processing.
P3.5	Embedding Extraction	Uses ArcFace (InsightFace) to generate a 512-dimensional face embedding vector from the detected face crop.
P3.6	Identity Matching	Compares the embedding against the local registered embeddings database using cosine similarity. Applies a majority-vote mechanism across several frames to confirm identity before logging.
P3.7	Attendance Submission	POSTs the confirmed employee identity and timestamp to POST /api/logs on the web API, secured with the FACIAL_RECOGNITION_API_KEY header.

The following data stores (database tables) are referenced across all DFD levels:

Label	Table Name	Contents
D1	personnel	Employee profiles: name, employee ID, department, position, vehicle plate, photo (Base64)
D2	accounts	Login credentials: username (= employee ID), hashed password, role (admin/user), linked personnel ID
D3	sessions	Active login tokens: session token, linked account ID, expiry timestamp
D4	attendance_logs	Attendance records: employee ID, name, department, vehicle plate, log type (TIME_IN/TIME_OUT), timestamp

- POST /api/auth/login — Validate credentials, create session, set cookie
- POST /api/auth/logout — Delete session, clear cookie
- GET /api/auth/me — Return current logged-in user info from cookie

Personnel Management (/api/personnel)

- GET /api/personnel — List all personnel (Admin) or own dept (User)
- POST /api/personnel — Create new personnel record + account (Admin only)
- GET /api/personnel/:id — Get specific personnel profile
- PUT /api/personnel/:id — Update personnel data (Admin only)
- DELETE /api/personnel/:id — Remove personnel + linked account (Admin only)

Attendance Logs (/api/logs)

- GET /api/logs — Retrieve recent attendance logs
- POST /api/logs — Receive detection from Python service (requires API key)
- GET /api/logs/personnel-photos — Export all personnel photos for face sync (requires API key)

Statistics (/api/stats)

- GET /api/stats — Aggregate total vs. present-today counts per department

Layer	Technology
Frontend	React 19 + Vite, TypeScript, TailwindCSS, React Query

Node.js + Express 5, TypeScript, Drizzle ORM



PostgreSQL (Replit-hosted)

Session-based (HTTP-only cookies, bcryptjs)



Python — InsightFace (RetinaFace + ArcFace buffalo_l model)

Face Tracking

DeepSORT

Preprocessing

CLAHE + Adaptive Gamma Correction (OpenCV)

RTSP streams via OpenCV (IP cameras)

Deployment

Replit (web app + API), Local machine (Python service)

